



005 Week 4 — English Worksheet (Advanced)

Topic: Petting Zoo — Using Helper Functions · **Course:** Theme Park Creator · **Time:** about 50 minutes

Your park has a market row and amenities. Now it needs a **petting zoo** — an outer fence, six cages inside, six different animals, and a feeding area.

The outer fence is a fence box. Every cage is a fence box. Same job, seven times, different sizes. **One** `build_fence` helper does all seven — and the whole zoo drops anywhere in your park from one number.

These are the blocks you use this week:

```
blocks.fill(OAK_FENCE, pos(x1, 0, z1), pos(x2, 0, z2), FillOperation.OUTLINE)
blocks.fill(HAY_BLOCK, pos(x1, 0, z1), pos(x2, 0, z1 + 1), FillOperation.REPLACE)
blocks.place(OAK_FENCE_GATE, pos(x1 + 9, 0, z1))
mobs.spawn(CHICKEN, pos(x1 + 1, 1, z1 + 1))
```

`FillOperation.OUTLINE` = the **edge** of the box only — the ground inside is left alone.

`FillOperation.REPLACE` = the **whole solid** box. `FillOperation.HOLLOW` = the shell, and the middle is **cleared to air**. `mobs.spawn(animal, pos(x, y, z))` puts **one** animal in **one** spot.

1 · How to Use a Helper Function

A **helper function** does one job. Functions above it **call** it. Functions above *those* call **them**.

Four steps, every time:

Step 1 — Find the job that repeats. Outer fence, six cages. One job, seven runs.

Step 2 — Write the job once, with a name that says the job: `build_fence`.

Step 3 — Put every changing part in the brackets. Material and four corners. Nothing fixed inside the helper.

Step 4 — Call it from wherever the job is needed. `build_cage` calls it. `build_zoo` calls it.

`build_zoo` also calls `build_cage` — a chain three levels deep.

```

build_zoo(0)
├─▶ build_fence(OAK_FENCE, 0, 18, 0, 12)    ← the outer fence
├─▶ build_cage(CHICKEN, OAK_FENCE, 2, 4, 2, 4)
│   └─▶ build_fence(OAK_FENCE, 2, 4, 2, 4)  ← the SAME helper
│       └─▶ spawn CHICKEN
│           (× 6 animals)
└─▶ build_feeding_area(14, 17, 4, 8)

```

Here is the helper. It takes **four corners**, so the caller decides where the box sits and how big it is:

```

def build_fence(fence, x1, x2, z1, z2):
    blocks.fill(fence, pos(x1, 0, z1), pos(x2, 0, z2), FillOperation.OUTLINE)

```

One helper runs seven times per zoo, at two different sizes. What would you have to change if every fence in the zoo had to become **BIRCH_FENCE** ?

2 · Predict 🧠

Read each function. Write what you think happens.

```
def build_cage(animal, fence, x1, x2, z1, z2):  
    build_fence(fence, x1, x2, z1, z2)  
    mobs.spawn(animal, pos(x1 + 1, 1, z1 + 1))  
  
build_cage(SHEEP, OAK_FENCE, 6, 8, 2, 4)
```

Which lines run, and in what order? Which value lands in `x2`, and which never reaches `build_fence` at all?


```
def build_zoo(x):  
    build_fence(OAK_FENCE, x, x + 18, 0, 12)  
    build_cage(CHICKEN, OAK_FENCE, x + 2, x + 4, 2, 4)  
    build_cage(COW, OAK_FENCE, x + 6, x + 8, 2, 4)  
    build_cage(PIG, OAK_FENCE, x + 10, x + 12, 2, 4)  
    build_feeding_area(x + 14, x + 17, 4, 8)  
  
build_zoo(0)  
build_zoo(40)
```

`build_zoo` is called twice. How many zoos are built, how many animals appear, and how many times does `build_fence` run?

3 · Spot the Bug 🐛

Each block is broken. Read what it should do, rewrite it so it works, then explain why the original was wrong.

Bug A — The outer fence should ring the zoo and leave everything inside it standing. Instead the cages **vanish** the moment the outer fence goes up, and six animals are left standing in the open.

```
def build_fence(fence, x1, x2, z1, z2):
    blocks.fill(fence, pos(x1, 0, z1), pos(x2, 0, z2), FillOperation.HOLLOW)

def build_zoo(x):
    build_cage(CHICKEN, OAK_FENCE, x + 2, x + 4, 2, 4)
    build_fence(OAK_FENCE, x, x + 18, 0, 12)

build_zoo(0)
```

Hint: **HOLLOW** clears the middle to air. **OUTLINE** leaves the middle alone. One word, in one place, fixes all seven fences.

Write the fixed code:

Why was it wrong? Why does your fix work?

Bug B — Six cages should stand in a row. Instead one cage appears, holding a **llama**, and the other five animals are nowhere.

```
def build_zoo(x):
    build_cage(CHICKEN, OAK_FENCE, x + 2, x + 4, 2, 4)
    build_cage(COW, OAK_FENCE, x + 2, x + 4, 2, 4)
    build_cage(PIG, OAK_FENCE, x + 2, x + 4, 2, 4)
    build_cage(LLAMA, OAK_FENCE, x + 2, x + 4, 2, 4)

build_zoo(0)
```

Hint: look at the corners each cage is given.

Write the fixed code:

Why was it wrong? Why does your fix work?

Bug C — The second zoo should stand far away from the first. Instead both zoos are built **on the same spot**, and only one zoo is there.

```
def build_zoo(x):
    build_fence(OAK_FENCE, 0, 18, 0, 12)
    build_cage(CHICKEN, OAK_FENCE, 2, 4, 2, 4)

build_zoo(0)
build_zoo(40)
```

Hint: the parameter x arrives, but nothing inside the function uses it.

Write the fixed code:

Why was it wrong? Why does your fix work?

4 · Build the Petting Zoo 🐑

In your homework world, write **one** helper and three functions that call down into it:

```
def build_fence(fence, x1, x2, z1, z2):
def build_cage(animal, fence, x1, x2, z1, z2):
def build_feeding_area(x1, x2, z1, z2):
def build_zoo(x):
```

`build_fence` builds one fence box with `FillOperation.OUTLINE`.

`build_cage` calls `build_fence`, then spawns its animal inside the box it just built.

`build_feeding_area` fills `HAY_BLOCK` for food and `WATER` for drink.

`build_zoo(x)` calls `build_fence` for the outer fence, places an `OAK_FENCE_GATE` in it, calls `build_cage` **six** times with **six different animals**, and calls `build_feeding_area` once.

Only `build_fence` may contain a fence `blocks.fill`. Every corner must come from a parameter — no fixed corner numbers inside the helper, so the whole zoo moves when `x` moves.

Animals: `CHICKEN` · `COW` · `PIG` · `SHEEP` · `RABBIT` · `LLAMA`

Cage corners inside an 18 × 12 zoo — front row `z 2→4`, back row `z 8→10`:

```
cages      x+2 → x+4      x+6 → x+8      x+10 → x+12
feeding    x+14 → x+17   z 4 → 8
gate       pos(x + 9, 0, 0)
```

Then build two zoos in different corners of your park:

```
build_zoo(0)
build_zoo(40)
```


5 · Show Your Work 📷 🎥

Record **one video** — one take, no stopping (a phone is fine). Show these in order:

1 · Your code. Scroll through it. Say what each part does.

2 · Your build. Point the camera. Name the parts.

Fill the blanks:

Today I built _____.

I built it using this code: _____.

In this code I used _____.

The hardest part was _____.

That part was hard because _____.

The most fun part was _____.

Something new I learned was _____.

Write your lines here, then say them in your video.

Submit ✓

Send this worksheet + your video to teacher on KakaoTalk.